1. INTRODUCTION

Temperature monitoring systems are widely used in industries, laboratories, homes, data centers, and safety applications to detect abnormal environmental conditions and prevent damage caused by overheating. Continuous monitoring of temperature is essential in many applications because sudden increases in temperature may lead to equipment failure, fire hazards, reduced system performance, and safety risks. Traditional methods of temperature monitoring often require manual observation, which can be time-consuming, inefficient, and less reliable in critical situations. Therefore, automated temperature monitoring systems have become increasingly important in modern embedded and IoT-based applications.

With the rapid development of embedded systems and Internet of Things (IoT) technologies, microcontrollers such as ESP32 are widely used for designing smart monitoring systems due to their high processing capability, low power consumption, and support for sensor interfacing. Sensors play an important role in collecting real-time environmental data, while microcontrollers process the sensor data and perform necessary actions automatically. In this project, the DHT22 temperature sensor is used to continuously monitor the surrounding temperature and send the measured values to the ESP32 microcontroller for processing and analysis.

The main objective of this project is to design and implement a Temperature Monitoring and Alert System using ESP32. The system continuously reads temperature values from the DHT22 sensor and compares them with a predefined threshold value programmed into the controller. Whenever the measured temperature exceeds the threshold limit, the ESP32 automatically activates a buzzer to alert the user about the abnormal temperature condition. The temperature readings are also displayed on the Serial Monitor for real-time monitoring and observation.

The proposed system provides a simple, efficient, reliable, and low-cost solution for temperature monitoring applications. The project helps in reducing manual monitoring effort and improving safety by generating immediate alerts during high-temperature conditions. The system can be used in various applications such as industrial monitoring systems, fire safety systems, electronic equipment protection, smart homes, and server room monitoring. The project is implemented using hardware components such as ESP32 microcontroller, DHT22

temperature sensor, buzzer, breadboard, jumper wires, and USB data cable. Arduino IDE is used for programming the ESP32 using Embedded C language.

The introduction chapter provides the overall background, objectives, scope, and significance of the project. It explains the importance of automated temperature monitoring systems and describes how the proposed project attempts to overcome the limitations of manual monitoring methods. The chapter also introduces the technologies and hardware components used in the implementation of the system and provides a strong foundation for the remaining chapters of the project report.

2. PROBLEM STATEMENT

Temperature plays a critical role in maintaining the safety and proper functioning of electronic devices, industrial equipment, laboratories, and residential environments. Sudden increases in temperature may lead to overheating, equipment damage, fire hazards, and reduced operational efficiency. In many places, temperature monitoring is still performed manually, which is time-consuming and less reliable, especially in situations where continuous observation is required. Manual monitoring methods also increase the possibility of human error and delayed response during abnormal temperature conditions.

2.1 Existing System & Limitations

> Existing temperature monitoring systems often lack real-time alert mechanisms, require continuous manual supervision, and may involve expensive or complex equipment. Delayed detection of abnormal temperature conditions can lead to equipment failure, fire hazards, safety risks, and operational losses.

The major limitations of the existing system include:

- Lack of real-time monitoring capabilities
- Dependency on manual supervision
- Delayed response to temperature changes
- Higher cost and complex infrastructure in advanced systems
- Possibility of human error during observation
- No automatic alert or notification mechanism
- Limited scalability for multiple monitoring points

2.2 Proposed System & Advantages

The proposed **Temperature Monitoring and Alert System** using **ESP32** is designed to overcome these limitations by providing continuous and automated temperature monitoring using a **DHT22 sensor** and **ESP32 microcontroller**.

Whenever the measured temperature exceeds a predefined threshold value, the system automatically activates a buzzer to alert the user immediately.

Advantages of the proposed system:

- Real-time temperature monitoring
- Automatic alert generation using a buzzer
- Reduced dependency on manual supervision
- Low-cost and simple hardware design
- Faster response to abnormal temperature conditions
- Suitable for multiple environments such as homes, industries, and server rooms

---

The proposed system provides an efficient, reliable, and cost-effective solution for real-time temperature monitoring applications. It improves safety, reduces manual effort, and ensures immediate response during critical temperature conditions. The system is particularly useful in environments where continuous monitoring is essential to prevent damage and ensure operational stability.

3. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY

The Requirement Analysis and Feasibility Study chapter explains the requirements necessary for the successful development and implementation of the Temperature Monitoring and Alert System using ESP32. Proper requirement analysis helps in understanding the operation of the system, identifying the required hardware and software resources, and ensuring reliable system performance. This chapter also helps in reducing errors during development and improving the overall efficiency of the project.

The proposed system is designed to continuously monitor environmental temperature using the DHT22 temperature sensor and generate an alert whenever the temperature exceeds a predefined threshold value. The ESP32 microcontroller acts as the processing unit, which receives sensor data, processes the information, and controls the buzzer output accordingly. The project uses simple and low-cost hardware components, making the system economical and easy to implement.

3.1 Functional Requirements

The functional requirements describe the major functions performed by the system. The Temperature Monitoring and Alert System performs the following functions:

- Continuous monitoring of environmental temperature
- Reading temperature data using DHT22 sensor
- Processing sensor data using ESP32 microcontroller
- Comparing temperature values with predefined threshold limit
- Activating buzzer alert when temperature exceeds threshold value
- Displaying temperature readings in Serial Monitor
- Real-time monitoring and alert generation

The system continuously accepts temperature data as input from the DHT22 sensor. The ESP32 processes the input data and performs the necessary operations based on the programmed conditions. If the measured temperature exceeds the safe limit, the buzzer is activated automatically to notify the user about the abnormal condition.

3.2 Hardware Requirements

Sl. No	Hardware Component	Purpose
1	ESP32 Microcontroller	Processes sensor data and controls system operation
2	DHT22 Temperature Sensor	Measures environmental temperature
3	Buzzer	Generates alert sound during high temperature
4	Breadboard	Used for circuit connections
5	Jumper Wires	Connects hardware components
6	USB Data Cable	Uploads code and powers ESP32
7	Power Supply	Provides electrical power to the system

Table 3.1: Hardware Centre Test Requirements

The ESP32 microcontroller acts as the main controller of the system. The DHT22 sensor continuously senses the temperature and sends the measured values to the ESP32. The buzzer acts as an output device and produces an alert sound whenever the temperature crosses the threshold limit.

3.3 Software Requirements

Sl. No	Software	Purpose
1	Arduino IDE	Writing and uploading program code
2	Embedded C	Programming language used for coding
3	ESP32 Board Package	Supports ESP32 programming in Arduino IDE
4	DHT Sensor Library	Enables communication with DHT22 sensor
5	Serial Monitor	Displays real-time temperature readings

Table 3.2: Software table Requirements

Arduino IDE is used for developing and uploading the program code to the ESP32 microcontroller. Embedded C language is used for implementing the control logic of the system. The DHT sensor library allows the ESP32 to communicate with the DHT22 sensor and read temperature values efficiently.

3.4 Feasibility Study

The proposed Temperature Monitoring and Alert System is technically feasible because all the required hardware and software components are easily available and compatible with each other. The project can be implemented using simple embedded system concepts and basic electronic components.

The system is economically feasible because it uses low-cost components such as ESP32, DHT22 sensor, and buzzer, making the overall implementation cost affordable. The project is also operationally feasible because the system is easy to operate, monitor, and maintain. The proposed solution provides reliable temperature monitoring and alert generation with minimal human intervention, making it suitable for practical real-time applications.

4. SYSTEM DESIGN

The System Design chapter explains the structure and working model of the proposed Temperature Monitoring and Alert System using ESP32. The chapter describes how different hardware and software components are organized and connected to perform continuous temperature monitoring and automatic alert generation. The system is designed to monitor environmental temperature using the DHT22 sensor and activate a buzzer whenever the temperature exceeds a predefined threshold value.

The proposed system consists of three major sections: input section, processing section, and output section. The DHT22 sensor acts as the input component by continuously sensing the environmental temperature. The ESP32 microcontroller acts as the processing unit, which receives the temperature data from the sensor and compares it with the programmed threshold value. The buzzer acts as the output device and generates an alert sound whenever the temperature exceeds the safe limit. The temperature readings are also displayed on the Serial Monitor for real-time monitoring and analysis.

The system is designed using simple embedded system architecture to ensure reliable operation, low cost, and easy implementation. The overall workflow of the system includes sensor data collection, data processing, threshold comparison, and alert generation. The system operates continuously and provides immediate alerts during abnormal temperature conditions.

4.1 System Architecture

The System Architecture explains the overall structure and interaction between the hardware and software components used in the project. The architecture of the proposed system mainly consists of DHT22 temperature sensor, ESP32 microcontroller, buzzer alert system, and Serial Monitor interface.

The DHT22 sensor continuously measures the surrounding temperature and sends the sensor readings to the ESP32 controller. The ESP32 processes the sensor data and compares the measured temperature value with the predefined threshold value stored in the program. If the temperature exceeds the threshold limit, the ESP32 activates the buzzer automatically to alert

the user. The measured temperature values are simultaneously displayed on the Serial Monitor for monitoring and analysis purposes.

The proposed system uses direct communication between the sensor and the ESP32 microcontroller. Since the project is designed as a standalone embedded monitoring system, cloud storage and external servers are not used in the current implementation. However, the system can be upgraded in future to support IoT-based cloud monitoring and mobile notifications.

Figure 4.1 shows the overall system architecture of the Temperature Monitoring and Alert System using ESP32.

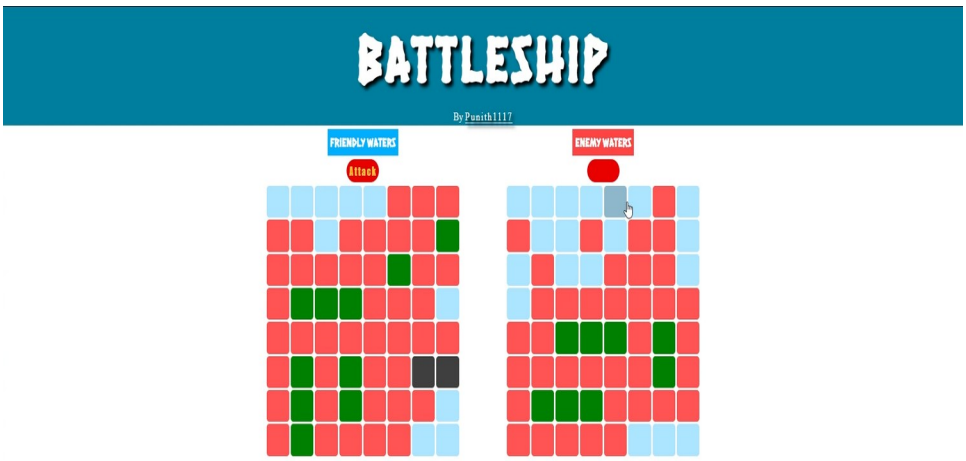

Figure 4.1: System Architecture Diagram

4.1.1 Component Specifications

Component	Parameter	Specification
DHT22	Voltage	3.3V to 5V
DHT22	Temperature Range	-40°C to 80°C
DHT22	Humidity Range	0% to 100%
ESP32	Operating Voltage	3.3V

Component	Parameter	Specification
ESP32	WiFi	802.11 b/g/n

Table 4.1: Component Specifications

The architecture diagram clearly shows the flow of data from the DHT22 sensor to the ESP32 microcontroller. The ESP32 processes the input data and controls the buzzer output based on the temperature threshold condition. The Serial Monitor displays the real-time temperature values continuously.

4.2 Data Flow Diagrams (DFD) / Flow Charts

This section explains the flow of information and operational sequence within the system. The Data Flow Diagram (DFD) and Flow Chart help in understanding how the system processes sensor data and generates output based on temperature conditions.

Level 0 DFD (Context Diagram)

The Level 0 DFD represents the overall interaction between the user and the Temperature Monitoring and Alert System.

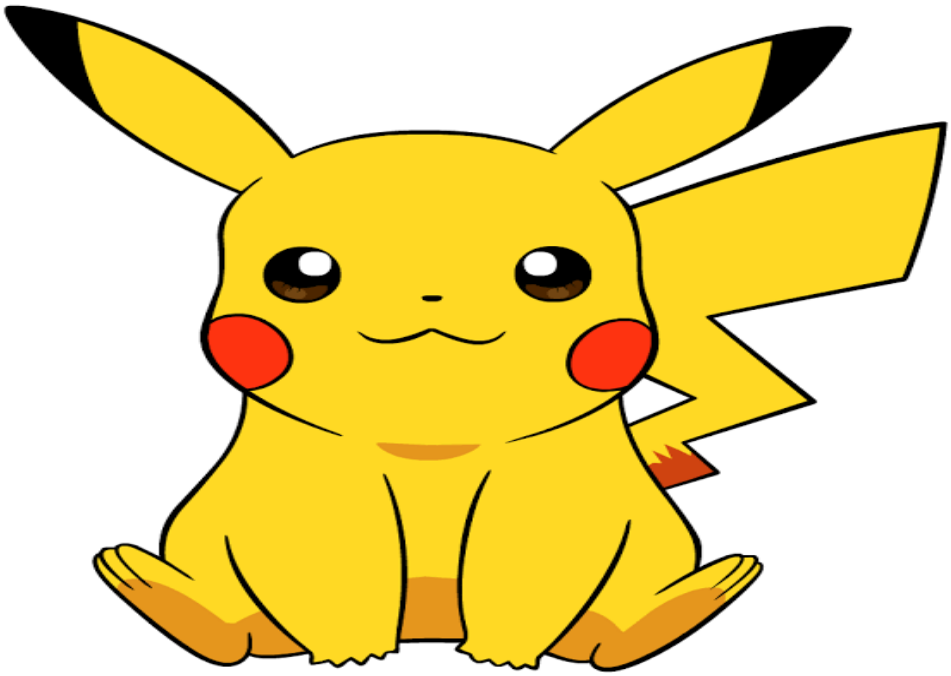

Figure 4.2: Level 0 Data Flow Diagram

The Level 0 DFD shows that the user interacts with the system through temperature monitoring and receives alert notifications whenever abnormal temperature conditions occur.

Level 1 DFD

The Level 1 DFD explains the internal processing steps of the proposed system.

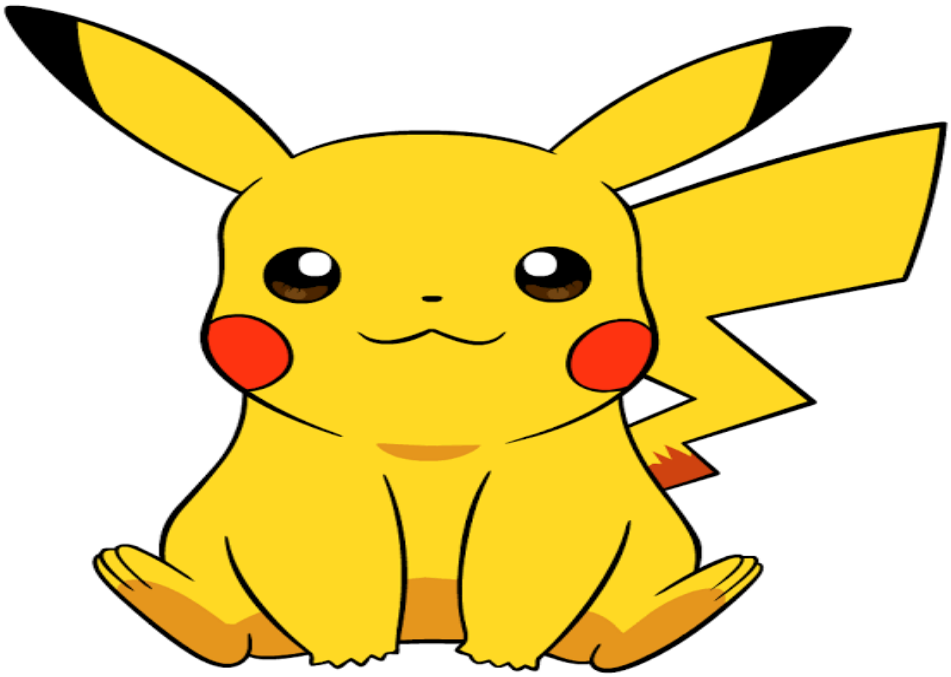

Figure 4.3: Level 1 Data Flow Diagram

The Level 1 DFD shows that the DHT22 sensor sends temperature data to the ESP32 controller. The ESP32 processes the data, compares it with the threshold value, displays the readings on the Serial Monitor, and activates the buzzer whenever the temperature exceeds the safe limit.

4.3 Flow Chart

The flow chart represents the complete operational sequence of the Temperature Monitoring and Alert System using ESP32.

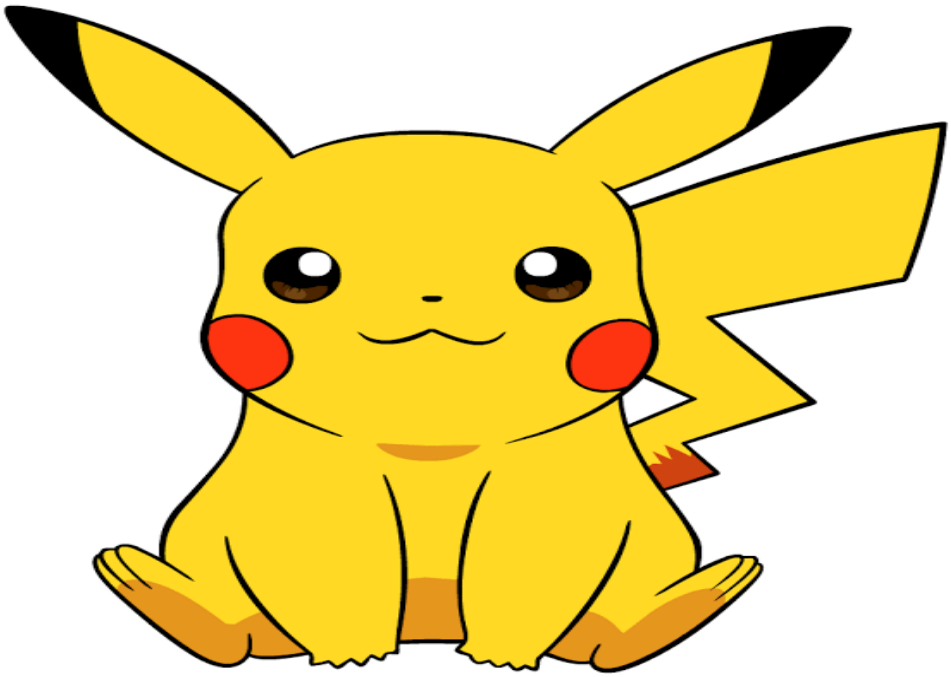

Figure 4.4: Flow Chart of Proposed System

The flow chart begins with system initialization. The DHT22 sensor continuously monitors the environmental temperature and sends the readings to the ESP32 controller. The ESP32 compares the temperature value with the predefined threshold condition. If the temperature exceeds the threshold value, the buzzer is activated automatically; otherwise, the buzzer remains OFF. The temperature readings are displayed continuously on the Serial Monitor, and the process repeats continuously for real-time monitoring.

5. IMPLEMENTATION

The Implementation chapter explains the practical development and execution of the proposed Temperature Monitoring and Alert System using ESP32. This chapter describes the hardware setup, software configuration, sensor interfacing, and program implementation used for developing the system. The implementation process involves connecting the DHT22 temperature sensor and buzzer to the ESP32 microcontroller and programming the controller using Arduino IDE.

The DHT22 sensor is used to continuously measure the surrounding temperature and send the sensor readings to the ESP32 controller. The ESP32 processes the received temperature data and compares it with the predefined threshold value programmed into the system. Whenever the temperature exceeds the threshold value, the ESP32 automatically activates the buzzer to alert the user. The temperature readings are also displayed continuously on the Serial Monitor for monitoring and analysis.

The project is implemented using simple hardware components and embedded programming concepts. Proper wiring connections are established between the ESP32, DHT22 sensor, and buzzer using jumper wires and breadboard. The system is powered through a USB data cable connected to the computer.

5.1 Hardware Implementation

The hardware implementation involves interfacing the DHT22 sensor and buzzer with the ESP32 microcontroller. The DHT22 sensor acts as the input device and continuously senses environmental temperature. The ESP32 microcontroller acts as the processing unit and controls the overall operation of the system. The buzzer acts as the output device and generates an alert sound whenever the temperature exceeds the threshold value.

Hardware Connections

Component	ESP32 Pin Connection
DHT22 VCC	3.3V
DHT22 DATA	GPIO4

Component	ESP32 Pin Connection
DHT22 GND	GND
Buzzer Positive	GPIO18
Buzzer Negative	GND

Table 5.1: Hardware Connections

The DHT22 sensor sends temperature readings to the ESP32 through GPIO4 pin. The ESP32 processes the temperature values and activates the buzzer connected to GPIO18 whenever the threshold condition is satisfied.

5.2 Software Configuration

The software implementation of the project is carried out using Arduino IDE. The ESP32 board package and DHT sensor library are installed in the Arduino IDE to support programming and communication with the sensor.

The following software configuration steps are performed during implementation:

- Installation of Arduino IDE
- Installation of ESP32 Board Package
- Installation of DHT Sensor Library
- Selection of ESP32 Board and COM Port
- Writing and uploading Embedded C program
- Serial Monitor configuration for temperature display

The Arduino IDE is used to write, compile, and upload the program code to the ESP32 microcontroller through the USB data cable.

5.3 Program Code

The program code is written using Embedded C language in Arduino IDE. The program continuously reads temperature values from the DHT22 sensor and compares them with the predefined threshold value. If the temperature exceeds the threshold limit, the buzzer is activated automatically.

Algorithm

1. Start the system
2. Initialize ESP32 and DHT22 sensor
3. Read temperature value from DHT22 sensor
4. Compare temperature with threshold value
5. If temperature exceeds threshold, activate buzzer
6. Otherwise keep buzzer OFF
7. Display temperature on Serial Monitor
8. Repeat the process continuously

Program Code

```
#include "DHT.h"
#define DHTPIN 4
#define DHTTYPE DHT22
#define BUZZER_PIN 18

DHT dht(DHTPIN, DHTTYPE);

float threshold = 35.0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("Temperature Monitoring Started");
}

void loop() {
  float temperature = dht.readTemperature();
  if (isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println(" °C");
  if (temperature > threshold) {
    digitalWrite(BUZZER_PIN, HIGH);
    Serial.println("ALERT! HIGH TEMPERATURE!");
  } else {
    digitalWrite(BUZZER_PIN, LOW);
  }
  delay(2000);
}
```

6. RESULT ANALYSIS

The Result Analysis chapter explains the performance and output obtained from the implementation of the Temperature Monitoring and Alert System using ESP32. The system was successfully tested under different temperature conditions to verify the proper operation of the DHT22 temperature sensor, ESP32 microcontroller, and buzzer alert mechanism. The project demonstrated reliable temperature monitoring and immediate alert generation whenever the temperature exceeded the predefined threshold value.

The DHT22 sensor continuously monitored the environmental temperature and transmitted the readings to the ESP32 controller. The ESP32 successfully processed the sensor data and compared the temperature values with the programmed threshold condition. During testing, whenever the temperature crossed the threshold limit, the buzzer was activated automatically to notify the user about the abnormal temperature condition. The system also displayed real-time temperature readings on the Serial Monitor continuously.

The project was tested under normal room temperature conditions as well as elevated temperature conditions using external heat sources. The observed results confirmed that the system responded correctly and generated alerts immediately after detecting high temperature values. The buzzer remained OFF during normal temperature conditions and turned ON automatically whenever the temperature exceeded the threshold value.

6.1 Results

The following observations were recorded during the testing of the system:

Sl. No	Temperature Condition	Buzzer Status	System Response
1	Temperature below threshold	OFF	Normal monitoring
2	Temperature equal to threshold	OFF	Continuous monitoring
3	Temperature above threshold	ON	Alert generated
4	Sensor disconnected	OFF	Error message displayed

Table 6.1: Result Observations

The obtained results indicate that the proposed system performs reliable temperature

monitoring and alert generation. The ESP32 controller successfully processed the temperature data from the DHT22 sensor and activated the buzzer whenever the threshold condition was satisfied.

6.2 Performance Analysis

The implemented system provides the following advantages:

- Continuous real-time temperature monitoring
- Immediate alert generation during abnormal conditions
- Low-cost implementation using simple hardware components
- Easy installation and operation
- Reliable and efficient system performance
- Reduced manual monitoring effort

The DHT22 sensor provided stable and accurate temperature readings throughout the testing process. The ESP32 microcontroller responded quickly to temperature changes and activated the buzzer without noticeable delay. The system consumed very low power and operated efficiently during continuous monitoring.

6.3 Snapshots

The following snapshots were captured during the implementation and testing of the project:

- Hardware setup of ESP32, DHT22 sensor, and buzzer
- Breadboard circuit connections
- Serial Monitor displaying temperature readings
- Buzzer activation during high temperature condition
- Arduino IDE program upload screen

Figure 6.1: Hardware Setup of Proposed System

Figure 6.2: Circuit Connection of ESP32 and DHT22

Figure 6.3: Serial Monitor Output

Figure 6.4: Buzzer Alert during High Temperature Detection

The successful execution and testing of the proposed system demonstrate that the Temperature Monitoring and Alert System using ESP32 provides an effective and reliable solution for real-time temperature monitoring applications.

7. CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The Temperature Monitoring and Alert System using ESP32 was successfully designed and implemented using embedded system and sensor interfacing concepts. The system continuously monitored environmental temperature using the DHT22 temperature sensor and generated an alert whenever the temperature exceeded the predefined threshold value. The ESP32 microcontroller successfully processed the sensor data and controlled the buzzer output efficiently.

The project demonstrated reliable real-time temperature monitoring and automatic alert generation. The DHT22 sensor provided accurate temperature readings, while the ESP32 controller responded quickly to abnormal temperature conditions. The buzzer alert mechanism helped in notifying the user immediately during high-temperature situations. The system also displayed real-time temperature readings on the Serial Monitor for continuous observation and analysis.

The proposed system provides a simple, low-cost, reliable, and efficient solution for temperature monitoring applications. The project helps in reducing manual monitoring effort and improving safety in environments where continuous temperature observation is necessary. The system can be effectively used in industries, laboratories, homes, server rooms, and electronic equipment protection systems.

7.2 LIMITATIONS AND FUTURE WORK

Although the proposed system performs reliable temperature monitoring and alert generation, there are certain limitations in the current implementation. The system currently operates as a standalone embedded monitoring system without remote monitoring capability. The project also uses only a buzzer alert mechanism and does not provide mobile or internet-based notifications.

Future Enhancements

The future enhancements of the project may include:

- Integration of Wi-Fi and IoT cloud platforms
- Mobile application for remote temperature monitoring
- SMS or email alert notifications
- LCD or OLED display for direct temperature display
- Cloud data storage and analysis
- Addition of smoke and gas sensors for fire detection systems
- Development of web-based monitoring dashboard
- Battery backup support for uninterrupted operation

The future improvements can make the system more advanced, intelligent, and suitable for large-scale industrial and smart monitoring applications. The project provides a strong foundation for developing advanced IoT-based environmental monitoring systems.

BIBLIOGRAPHY

[1] R. Sharma and K. Patel, “IoT Based Smart Agriculture Monitoring System,” *International Journal of Engineering Research & Technology (IJERT)*, vol. 12, no. 5, pp. 112–118, May 2023.

[2] O. Hersent, D. Boswarthick, and O. Elloumi, *The Internet of Things: Key Applications and Protocols*, 2nd ed. Hoboken, NJ, USA: Wiley, 2012.

[3] Arduino, “Arduino UNO Technical Specifications,” 2025. Available: Arduino Official Website.

[4] M. Singh, “Wireless Sensor Networks for IoT Applications,” *Journal of Embedded Systems and Applications*, vol. 8, no. 2, pp. 78–84, Feb. 2022.

[5] A. Verma and S. Nair, “Cloud-Based IoT Monitoring and Control System,” in *Proceedings of the International Conference on Smart Computing and Communication*, Bangalore, India, 2023, pp. 210–215.

[6] Python Software Foundation, “Python Programming Language Documentation,” 2025. Available: Python Official Website.

[7] A. Bahga and V. Madisetti, *Internet of Things: A Hands-On Approach*. Hyderabad, India: Universities Press, 2015.

[8] Raspberry Pi Foundation, “Raspberry Pi Documentation,” 2025. Available: Raspberry Pi Official Website.

[9] M. Kranz, *Building the Internet of Things: Implement New Business Models, Disrupt Competitors, Transform Your Industry*. Hoboken, NJ, USA: Wiley, 2016.

[10] S. Kumar and P. Ramesh, “Smart Home Automation Using IoT,” *International Journal of Advanced Research in Computer Science*, vol. 10, no. 3, pp. 45–50, Mar. 2024.